

NAME:- U.PRANEETH

ROLL NO:- 2520030188

SEC NO:- 2520030188

```
DROP DATABASE IF EXISTS BankDB1;  
CREATE DATABASE BankDB1;  
USE BankDB1;
```

Output



Action Output

	#	Time	Action
✓	1	20:15:47	DROP DATABASE IF EXISTS BankDB1
✓	2	20:15:47	CREATE DATABASE BankDB1
✓	3	20:15:47	USE BankDB1

```
CREATE TABLE Customer(  
    Customer_ID INT PRIMARY KEY,  
    Customer_Name VARCHAR(100) NOT NULL,  
    Phone VARCHAR(15),  
    Email VARCHAR(100),  
    City VARCHAR(50)  
);
```



4 20:15:47 CREATE TABLE Customer(Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR

```
CREATE TABLE Account(  
    Account_No INT PRIMARY KEY,  
    Customer_ID INT NOT NULL,  
    Account_Type VARCHAR(20) NOT NULL,  
    Balance DECIMAL(12,2) DEFAULT 0 CHECK(Balance>=0),  
    Branch VARCHAR(50) NOT NULL,  
    FOREIGN KEY(Customer_ID) REFERENCES Customer(Customer_ID)  
);
```



5 20:15:48 CREATE TABLE Account(Account_No INT PRIMARY KEY, Customer_ID INT NOT NULL

```
CREATE TABLE Bank_Transaction(  
    Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Account_No INT NOT NULL,  
    Transaction_Type VARCHAR(20) NOT NULL,  
    Amount DECIMAL(12,2) NOT NULL CHECK(Amount>0),  
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY(Account_No) REFERENCES Account(Account_No),  
    CHECK(Transaction_Type IN('DEPOSIT','WITHDRAW'))  
);
```



6 20:15:48 CREATE TABLE Bank_Transaction(Transaction_ID INT PRIMARY KEY AUTO_INCREMENT

```

CREATE TABLE Loan(
    Loan_ID INT PRIMARY KEY,
    Customer_ID INT NOT NULL,
    Loan_Type VARCHAR(30),
    Loan_Amount DECIMAL(12,2) CHECK(Loan_Amount>0),
    Interest_Rate DECIMAL(5,2) CHECK(Interest_Rate>=0),
    FOREIGN KEY(Customer_ID) REFERENCES Customer(Customer_ID)
);

```

7 20:15:48 CREATE TABLE Loan(Loan_ID INT PRIMARY KEY, Customer_ID INT NOT NULL,

```

CREATE TABLE Transaction_Audit(
    Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
    Transaction_ID INT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
);

```

8 20:15:48 CREATE TABLE Transaction_Audit(Audit_ID INT PRIMARY KEY AUTO_INCREMENT,

```

CREATE TABLE Account_Audit(
    Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
    Account_No INT,
    Old_Balance DECIMAL(12,2),
    New_Balance DECIMAL(12,2),
    Changed_Date DATETIME DEFAULT CURRENT_TIMESTAMP
);

```



9 20:15:48 CREATE TABLE Account_Audit(Audit_ID INT PRIMARY KEY AUTO_INCREMENT, A

INSERT INTO Customer VALUES

```
(101,'Ravi Kumar','9876543210','ravi@gmail.com','Hyderabad'),  
(102,'Priya Sharma','9876543211','priya@gmail.com','Vijayawada'),  
(103,'Arjun Reddy','9876543212','arjun@gmail.com','Bangalore'),  
(104,'Sneha Rao','9876543213','sneha@gmail.com','Chennai'),  
(105,'Kiran Kumar','9876543214','kiran@gmail.com','Hyderabad'),  
(106,'Anita Desai','9876543215','anita@gmail.com','Mumbai'),  
(107,'Rahul Verma','9876543216','rahul@gmail.com','Delhi'),  
(108,'Neha Singh','9876543217','neha@gmail.com','Pune'),  
(109,'Vikram Patil','9876543218','vikram@gmail.com','Bangalore'),
```



10 20:15:48 INSERT INTO Customer VALUES (101,'Ravi Kumar','9876543210','ravi@gmail.com','Hyde

INSERT INTO Account VALUES

```
(10001,101,'Savings',50000,'Hyderabad'),  
(10002,102,'Savings',75000,'Vijayawada'),  
(10003,103,'Current',120000,'Bangalore'),  
(10004,104,'Savings',45000,'Chennai'),  
(10005,105,'Current',90000,'Hyderabad'),  
(10006,106,'Savings',35000,'Mumbai'),  
(10007,107,'Current',200000,'Delhi'),  
(10008,108,'Savings',15000,'Pune'),  
(10009,109,'Current',85000,'Bangalore'),
```

p

Result Grid				Filter Rows:		Edit:				Export/Import:		
	Audit_ID	Account_No	Old_Balance	New_Balance	Changed_Date							
▶	1	10001	50000.00	60000.00	2026-08-27 20:15:48							
	2	10002	75000.00	90000.00	2026-08-27 20:15:48							
	3	10003	120000.00	100000.00	2026-08-27 20:15:48							
	4	10004	45000.00	50000.00	2026-08-27 20:15:48							
	5	10005	90000.00	80000.00	2026-08-27 20:15:48							
	6	10006	35000.00	60000.00	2026-08-27 20:15:48							
	7	10007	200000.00	150000.00	2026-08-27 20:15:48							
	8	10008	15000.00	27000.00	2026-08-27 20:15:48							
	9	10009	85000.00	80000.00	2026-08-27 20:15:48							
	10	10010	60000.00	90000.00	2026-08-27 20:15:48							
	11	10001	60000.00	58000.00	2026-08-27 20:15:48							
	12	10002	90000.00	87000.00	2026-08-27 20:15:48							
Result 26		Result 27	Result 28	Result 29	Result 30	Customer 31	Acc					

Output

Action Output

#	Time	Action
✓ 7	20:15:48	CREATE TABLE Loan(Loan_ID INT PRIMARY KEY, Customer_ID INT NOT NU
✓ 8	20:15:48	CREATE TABLE Transaction_Audit(Audit_ID INT PRIMARY KEY AUTO_INCREM
✓ 9	20:15:48	CREATE TABLE Account_Audit(Audit_ID INT PRIMARY KEY AUTO_INCREMENT
✓ 10	20:15:48	INSERT INTO Customer VALUES (101,'Ravi Kumar','9876543210','ravi@gmail.com','H
✓ 11	20:15:48	INSERT INTO Account VALUES (10001,101,'Savings',50000,'Hyderabad'), (10002,10

INSERT INTO Loan VALUES

```
(501,101,'Home Loan',5000000,7.5),
(502,102,'Education Loan',1000000,6.5),
(503,103,'Car Loan',800000,8.2),
(504,104,'Personal Loan',500000,10.5),
(505,107,'Business Loan',2000000,9.0);
```

✓ 12 20:15:48 INSERT INTO Loan VALUES (501,101,'Home Loan',5000000,7.5), (502,102,'Education Loan',1000000,6.5), (503,103,'Car Loan',800000,8.2), (504,104,'Personal Loan',500000,10.5), (505,107,'Business Loan',2000000,9.0);


```

--
92      -- ===== TRIGGERS =====
93
94      DELIMITER //
95
96      • CREATE TRIGGER CheckBalance
97        BEFORE UPDATE ON Account
98        FOR EACH ROW
99        BEGIN
100        IF NEW.Balance<0 THEN

```

```

105
106      • CREATE TRIGGER BeforeTransaction
107        BEFORE INSERT ON Bank_Transaction
108        FOR EACH ROW
109        BEGIN
110            DECLARE b DECIMAL(12,2);
111
112        IF NOT EXISTS(

```

```

112        IF NOT EXISTS(
113            SELECT 1 FROM Account WHERE Account_No=NEW.Account_No
114        ) THEN
115            SIGNAL SQLSTATE '45000'
116            SET MESSAGE_TEXT='Account does not exist';
117        END IF;
118
119        IF NEW.Amount<=0 THEN
120            SIGNAL SQLSTATE '45000'
121            SET MESSAGE_TEXT='Invalid amount';

```

```
121         SET MESSAGE_TEXT='Invalid amount';
122     END IF;
123
124     IF NEW.Transaction_Type='WITHDRAW' THEN
125         SELECT Balance INTO b
126         FROM Account
127         WHERE Account_No=NEW.Account_No;
128
129         IF NEW.Amount>b THEN
130             SIGNAL SQLSTATE '45000'
```

```
130             SIGNAL SQLSTATE '45000'
131             SET MESSAGE_TEXT='Insufficient balance';
132         END IF;
133     END IF;
134 END//
135
136 • CREATE TRIGGER AfterTransaction
137 AFTER INSERT ON Bank_Transaction
138 FOR EACH ROW
139 BEGIN
```

```
139 BEGIN
140     INSERT INTO Transaction_Audit
141     (Transaction_ID,Account_No,Transaction_Type,Amount)
142     VALUES
143     (NEW.Transaction_ID,NEW.Account_No,
144     NEW.Transaction_Type,NEW.Amount);
145
146     IF NEW.Transaction_Type='DEPOSIT' THEN
147         UPDATE Account
148         SET Balance=Balance+NEW.Amount
149         WHERE Account_No=NEW.Account_No;
150     ELSE
151         UPDATE Account
152         SET Balance=Balance-NEW.Amount
153         WHERE Account_No=NEW.Account_No;
154     END IF;
155 END//
156
157 • CREATE TRIGGER PreventCustomerDelete
158 BEFORE DELETE ON Customer
159 FOR EACH ROW
160 BEGIN
161     IF EXISTS(
```



```
163         WHERE Customer_ID=OLD.Customer_ID
164     ) THEN
165         SIGNAL SQLSTATE '45000'
166         SET MESSAGE_TEXT='Customer has an account';
167     END IF;
168 END//
169
170 ● CREATE TRIGGER AccountAudit
171     AFTER UPDATE ON Account
172     FOR EACH ROW
173     BEGIN
174         IF OLD.Balance<>NEW.Balance THEN
175             INSERT INTO Account_Audit
176             (Account_No,Old_Balance,New_Balance)
177             VALUES
178             (NEW.Account_No,OLD.Balance,NEW.Balance);
179         END IF;
180     END//
181
182 ● CREATE TRIGGER PreventAccountDelete
183     BEFORE DELETE ON Account
184     FOR EACH ROW
185     BEGIN
```

```

182 • CREATE TRIGGER PreventAccountDelete
183 BEFORE DELETE ON Account
184 FOR EACH ROW
185 BEGIN
186     IF EXISTS(
187         SELECT 1 FROM Bank_Transaction
188         WHERE Account_No=OLD.Account_No
189     ) THEN
190         SIGNAL SQLSTATE '45000'
191         SET MESSAGE_TEXT='Account has transactions';
192     END IF;
193 END//
194
195 DELIMITER ;
196

```

	#	Time	Action
✓	13	20:52:07	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR E
✓	14	20:52:07	CREATE TRIGGER BeforeTransaction BEFORE INSERT ON Bank_Transa
✓	15	20:52:07	CREATE TRIGGER AfterTransaction AFTER INSERT ON Bank_Transaction
✓	16	20:52:07	CREATE TRIGGER PreventCustomerDelete BEFORE DELETE ON Custom
✓	17	20:52:07	CREATE TRIGGER AccountAudit AFTER UPDATE ON Account FOR EAC
✓	18	20:52:07	CREATE TRIGGER PreventAccountDelete BEFORE DELETE ON Account

```
INSERT INTO Bank_Transaction(Account_No,Transaction_Type,Amount) VALUES
(10001,'DEPOSIT',10000),
(10002,'DEPOSIT',15000),
(10003,'WITHDRAW',20000),
(10004,'DEPOSIT',5000),
(10005,'WITHDRAW',10000),
(10006,'DEPOSIT',25000),
(10007,'WITHDRAW',50000),
(10008,'DEPOSIT',12000),
(10009,'WITHDRAW',5000),
```



19 20:52:08 -- ===== TRANSACTIONS ===== INSERT INTO Bank_Tra

```
219      -- ===== PROCEDURES =====
220
221      DELIMITER //
222
223      ● CREATE PROCEDURE GetBranchAccounts(IN p VARCHAR(50))
224      ○ BEGIN
225          SELECT * FROM Account WHERE Branch=p;
226      END//
227
228      ● CREATE PROCEDURE GetCustomerDetails(IN p INT)
229      ○ BEGIN
230          SELECT C.*,A.Account_No,A.Account_Type,A.Balance,A.Branch
231          FROM Customer C JOIN Account A
232          ON C.Customer_ID=A.Customer_ID
233          WHERE C.Customer_ID=p;
234      END//
235
236      ● CREATE PROCEDURE GetTotalBalance(IN p INT)
237      ○ BEGIN
238          SELECT C.Customer_ID,C.Customer_Name,
239              COALESCE(SUM(A.Balance),0) Total_Balance
240          FROM Customer C LEFT JOIN Account A
```

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

SQL File 8*

```
242 WHERE C.Customer_ID=p
243 GROUP BY C.Customer_ID,C.Customer_Na
244 END//
245
246 • CREATE PROCEDURE HighBalance(IN p DECIMAL
247 BEGIN
248 SELECT * FROM Account
249 WHERE Balance>=p
250 ORDER BY Balance DESC;
251 END//
252
253 • CREATE PROCEDURE TransferMoney(
254 IN s INT,IN r INT,IN amt DECIMAL(12,
255 )
256 BEGIN
257 DECLARE b DECIMAL(12,2);
258
259 DECLARE EXIT HANDLER FOR SQLEXCEPTION
260 BEGIN
261 ROLLBACK;
262 RESIGNAL;
263 END;
264
```

ns
ges
m Variables

store

own

ports
hema Setup

mas

```
263      END;  
264  
265      IF amt<=0 THEN  
266          SIGNAL SQLSTATE '45000'  
267          SET MESSAGE_TEXT='Invalid amount';  
268      END IF;  
269  
270      IF s=r THEN  
271          SIGNAL SQLSTATE '45000'  
272          SET MESSAGE_TEXT='Same account';  
273      END IF;  
274  
275      IF NOT EXISTS(SELECT 1 FROM Account WHERE Account_No=  
276          SIGNAL SQLSTATE '45000'  
277          SET MESSAGE_TEXT='Sender not found';  
278      END IF;  
279  
280      IF NOT EXISTS(SELECT 1 FROM Account WHERE Account_No=  
281          SIGNAL SQLSTATE '45000'  
282          SET MESSAGE_TEXT='Receiver not found';  
283      END IF;  
284  
285      START TRANSACTION;
```


les

ip

```
284
285     START TRANSACTION;
286
287     SELECT Balance INTO b
288     FROM Account
289     WHERE Account_No=s
290     FOR UPDATE;
291
292     IF b<amt THEN
293         SIGNAL SQLSTATE '45000'
294         SET MESSAGE_TEXT='Insufficient balance';
295     END IF;
296
297     INSERT INTO Bank_Transaction
298     (Account_No,Transaction_Type,Amount)
299     VALUES(s,'WITHDRAW',amt);
300
301     INSERT INTO Bank_Transaction
302     (Account_No,Transaction_Type,Amount)
303     VALUES(r,'DEPOSIT',amt);
304
305     COMMIT;
306 END//
```

ables

ietup

```
308 • CREATE PROCEDURE LoanInterest(  
309     IN amt DECIMAL(12,2),  
310     IN rate DECIMAL(5,2),  
311     IN yrs INT  
312 )  
313 BEGIN  
314     IF amt<=0 OR rate<0 OR yrs<=0 THEN  
315         SIGNAL SQLSTATE '45000'  
316         SET MESSAGE_TEXT='Invalid loan details';  
317     END IF;  
318  
319     SELECT amt*rate*yrs/100 Simple_Interest,  
320            amt+(amt*rate*yrs/100) Total_Amount;  
321 END//  
322  
323 • CREATE PROCEDURE Top5()  
324 BEGIN  
325     SELECT * FROM Account  
326     ORDER BY Balance DESC LIMIT 5;  
327 END//  
328  
329 • CREATE PROCEDURE BranchCount()  
330 BEGIN
```

es

p

```

330 BEGIN
331     SELECT Branch,COUNT(*) Number_of_Accounts
332     FROM Account
333     GROUP BY Branch;
334 END//
335
336 ● CREATE PROCEDURE HighLoans(IN p DECIMAL(12,2))
337 BEGIN
338     SELECT C.Customer_ID,C.Customer_Name,
339           L.Loan_Type,L.Loan_Amount,L.Interest_Rate
340     FROM Customer C JOIN Loan L
341     ON C.Customer_ID=L.Customer_ID
342     WHERE L.Loan_Amount>p
343     ORDER BY L.Loan_Amount DESC;
344 END//
345
346 ● CREATE PROCEDURE TransactionHistory(IN p INT)
347 BEGIN
348     SELECT * FROM Bank_Transaction
349     WHERE Account_No=p
350     ORDER BY Transaction_ID DESC;
351 END//

```

```

352
353 • CREATE PROCEDURE AllAccounts()
354 BEGIN
355     SELECT A.Account_No,C.Customer_ID,C.Customer_Name,
356           C.Phone,C.Email,A.Account_Type,
357           A.Balance,A.Branch
358     FROM Account A JOIN Customer C
359     ON A.Customer_ID=C.Customer_ID
360     ORDER BY A.Account_No;
361 END//
362

```

✓	19	21:01:55	-- ===== TRANSACTIONS ===== INSERT INTO Ban
✓	20	21:01:55	CREATE PROCEDURE GetBranchAccounts(IN p VARCHAR(50)) BEGIN SELECT
✓	21	21:01:55	CREATE PROCEDURE GetCustomerDetails(IN p INT) BEGIN SELECT C.*,A.Acc
✓	22	21:01:55	CREATE PROCEDURE GetTotalBalance(IN p INT) BEGIN SELECT C.Customer,
✓	23	21:01:55	CREATE PROCEDURE HighBalance(IN p DECIMAL(12,2)) BEGIN SELECT * FR
✓	24	21:01:55	CREATE PROCEDURE TransferMoney(IN s INT,IN r INT,IN amt DECIMAL(12,2)

✓	24	21:01:55	CREATE PROCEDURE TransferMoney(IN s INT,IN r INT,IN amt DECIMAL(12,2)
✓	25	21:01:55	CREATE PROCEDURE LoanInterest(IN amt DECIMAL(12,2), IN rate DECIMA
✓	26	21:01:55	CREATE PROCEDURE Top5() BEGIN SELECT * FROM Account ORDER BY
✓	27	21:01:55	CREATE PROCEDURE BranchCount() BEGIN SELECT Branch,COUNT(*) Numb
✓	28	21:01:55	CREATE PROCEDURE HighLoans(IN p DECIMAL(12,2)) BEGIN SELECT C.Cus
✓	29	21:01:55	CREATE PROCEDURE TransactionHistory(IN p INT) BEGIN SELECT * FROM B

✓	30	21:01:56	CREATE PROCEDURE AllAccounts() BEGIN SELECT A.Account_No,C.Customer_ID,C.C
---	----	----------	--

```

365 • -- ===== TEST =====
366
367 CALL GetBranchAccounts('Hyderabad');
368 • CALL GetCustomerDetails(101);
369 • CALL GetTotalBalance(101);
370 • CALL HighBalance(100000);
371
372 • CALL TransferMoney(10001,10002,5000);
373
374 • CALL LoanInterest(5000000,7.5,10);

```

```

375
376 • CALL Top5();
377 • CALL BranchCount();
378 • CALL HighLoans(1000000);
379 • CALL TransactionHistory(10001);
380 • CALL AllAccounts();
381

```

✓	31	21:01:56	-- ===== TEST ===== CALL GetBranchAccounts(Hyderabad)
✓	32	21:01:56	CALL GetCustomerDetails(101)
✓	33	21:01:56	CALL GetTotalBalance(101)
✓	34	21:01:56	CALL HighBalance(100000)
✓	35	21:01:56	CALL TransferMoney(10001,10002,5000)
✓	36	21:01:56	CALL LoanInterest(5000000,7.5,10)

✓	37	21:01:56	CALL Top5()
✓	38	21:01:56	CALL BranchCount()
✓	39	21:01:56	CALL HighLoans(1000000)
✓	40	21:01:56	CALL TransactionHistory(10001)
✓	41	21:01:56	CALL AllAccounts()

ibles

```
380 • CALL AllAccounts();
381
382 -- ===== CHECK =====
383
384 • SELECT * FROM Customer;
385 • SELECT * FROM Account;
386 • SELECT * FROM Loan;
387 • SELECT * FROM Bank_Transaction;
388 • SELECT * FROM Transaction_Audit;
389 • SELECT * FROM Account_Audit;
```

	#	Time	Action
✓	42	21:01:56	SELECT * FROM Customer LIMIT 0, 1000
✓	43	21:01:56	SELECT * FROM Account LIMIT 0, 1000
✓	44	21:01:57	SELECT * FROM Loan LIMIT 0, 1000
✓	45	21:01:57	SELECT * FROM Bank_Transaction LIMIT 0, 1000
✓	46	21:01:57	SELECT * FROM Transaction_Audit LIMIT 0, 1000
✓	47	21:01:57	SELECT * FROM Account_Audit LIMIT 0, 1000

p

Result Grid					
Filter Rows:					
Edit:					
Export/Import:					
	Customer_ID	Customer_Name	Phone	Email	City
▶	101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
	102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
	103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
	104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
	105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
	106	Anita Desai	9876543215	anita@gmail.com	Mumbai
	107	Rahul Verma	9876543216	rahul@gmail.com	Delhi
	108	Neha Singh	9876543217	neha@gmail.com	Pune
	109	Vikram Patil	9876543218	vikram@gmail.com	Bangalore
	110	Sonia Gupta	9876543219	sonia@gmail.com	Hyderabad
•	NULL	NULL	NULL	NULL	NULL

386 • `SELECT * FROM Loan;`

Result Grid						Filter Rows:	Edit:	Export/Import:	
	Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate				
▶	501	101	Home Loan	5000000.00	7.50				
	502	102	Education Loan	1000000.00	6.50				
	503	103	Car Loan	800000.00	8.20				
	504	104	Personal Loan	500000.00	10.50				
	505	107	Business Loan	2000000.00	9.00				
•	NULL	NULL	NULL	NULL	NULL				

Result Grid						Filter Rows:	Edit:	Export/Import:	
	Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date				
▶	1	10001	DEPOSIT	10000.00	2026-08-27 21:01:55				
	2	10002	DEPOSIT	15000.00	2026-08-27 21:01:55				
	3	10003	WITHDRAW	20000.00	2026-08-27 21:01:55				
	4	10004	DEPOSIT	5000.00	2026-08-27 21:01:55				
	5	10005	WITHDRAW	10000.00	2026-08-27 21:01:55				
	6	10006	DEPOSIT	25000.00	2026-08-27 21:01:55				
	7	10007	WITHDRAW	50000.00	2026-08-27 21:01:55				
	8	10008	DEPOSIT	12000.00	2026-08-27 21:01:55				
	9	10009	WITHDRAW	5000.00	2026-08-27 21:01:55				
	10	10010	DEPOSIT	30000.00	2026-08-27 21:01:55				
	11	10001	WITHDRAW	2000.00	2026-08-27 21:01:55				
	12	10002	WITHDRAW	3000.00	2026-08-27 21:01:55				

Bank Transaction 23

✓	48	21:06:06	SELECT * FROM Customer LIMIT 0, 1000
✓	49	21:06:09	SELECT * FROM Account LIMIT 0, 1000
✓	50	21:06:11	SELECT * FROM Loan LIMIT 0, 1000
✓	51	21:06:16	SELECT * FROM Customer LIMIT 0, 1000
✓	52	21:06:30	SELECT * FROM Account LIMIT 0, 1000
✓	53	21:06:37	SELECT * FROM Loan LIMIT 0, 1000

✓	53	21:06:37	SELECT * FROM Loan LIMIT 0, 1000
✓	54	21:06:50	SELECT * FROM Bank_Transaction LIMIT 0, 1000
✓	55	21:07:06	SELECT * FROM Transaction_Audit LIMIT 0, 1000
✓	56	21:07:13	SELECT * FROM Transaction_Audit LIMIT 0, 1000